Image Classification with CNN: Project Report

For this project, I chose a dataset of 28,000 medium-quality animal images belonging to ten categories: dog, cat, horse, spider, butterfly, chicken, sheep, cow, squirrel, elephant.
The aim of the project was to create a CNN that would accept an image as input and accurately classify it into one of the ten categories.
I also aimed to create a small Flask application with a UI to let a user upload an image and get a prediction with my model.

Data Preprocessing

I inspected my data to make sure it is valid and balanced. I plotted the first nine images from my dataset to visually validate it and make sure the labels were correct.

I also checked the number of images belonging to each class:

```
butterfly            : 1723
cat                  : 1345
chicken              : 2476
cow                  : 1484
dog                  : 3915
elephant             : 1152
horse                : 2087
sheep                : 1438
spider               : 3842
squirrel             : 1482
```

I noticed that the classes were unbalanced - for example, the “dog” class contained almost four times as many images as the “elephant” class. If not accounted for, this would mean that my model would be much better at recognizing dogs than elephants.
To account for this class imbalance, I calculated the weights for each class:

```
from sklearn.utils.class_weight import compute_class_weight

# Calculate weights
class_weights = compute_class_weight(
   class_weight='balanced',
   classes=np.unique(labels),
   y=labels
)
class_weight_dict = dict(enumerate(class_weights))
```

I will use these later when fitting the model.

Finally, I normalized the data to ensure my pixel values are between 0 and 1:

```
def normalize_img(image, label):
   '''Function scales pixels to [0, 1]'''
   return tf.cast(image, tf.float32) / 255.0, label

# Apply function to training and validation datasets
train_ds = train_ds.map(normalize_img)
val_ds = val_ds.map(normalize_img)
```

Model Architecture

I started with a basic architecture of three convolutional layers. I used the Adam optimizer and a small learning rate of 0.0001.

```
model = models.Sequential(
   [
      layers.Input(shape=(224, 224, 3)),
      layers.Conv2D(16, kernel_size=(3, 3), activation='relu'),
      layers.MaxPooling2D(pool_size=(2, 2)),

      layers.Conv2D(32, kernel_size=(3, 3), activation='relu'),
```

```
        layers.MaxPooling2D(pool_size=(2, 2)),

        layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
        layers.MaxPooling2D(pool_size=(2, 2)),

        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(10, activation="softmax")
    ]
)
```

I suspected that the model would overfit, so I added a dropout layer at the end and utilized early stopping to prevent it. However, turns out that wasn't enough - the model was giving me over 80% accuracy on the training set, with a little over 50% on the validation set.
This indicates strong overfitting.

To fix this issue, I decided to try to diversify my dataset by utilizing Data Augmentation. I added random flip, rotation, zoom, and contrast. I also decided to add Batch Normalization after each layer to make the learning process more stable.
In addition, I replaced Flatten with Global Average Pooling to reduce the total number of parameters to help the network focus on recognizing particular features instead of their specific location.

```
model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),

    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.15),
    layers.RandomContrast(0.1),

    # Block 1 - 32 filters
    layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    # Block 2 - 64 filters
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
```

```
    layers.MaxPooling2D((2, 2)),

    # Block 3 - 128 filters
    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax")
])
```

This model gave me a consistent result of around 60% accuracy on both the training and the validation set. No more overfitting!
However, the result was still significantly lower than I would like, so I began trying to figure out how to make my model more accurate.

I implemented the following changes:

- Added two more blocks of 256 filters each - my previous model wasn't deep enough to handle the complex features of ten different animal species
- Moved Activation after BatchNormalization - to make sure data is centered and scaled before being transformed
- Reduced the arguments in RandomRotation and RandomZoom from 0.15 to 0.05 - if they are too high, the model might be seeing noisy images

After some additional experimentation, I came up with this final architecture:

```
model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),

    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05),
    layers.RandomZoom(0.05),
    layers.RandomContrast(0.1),
```

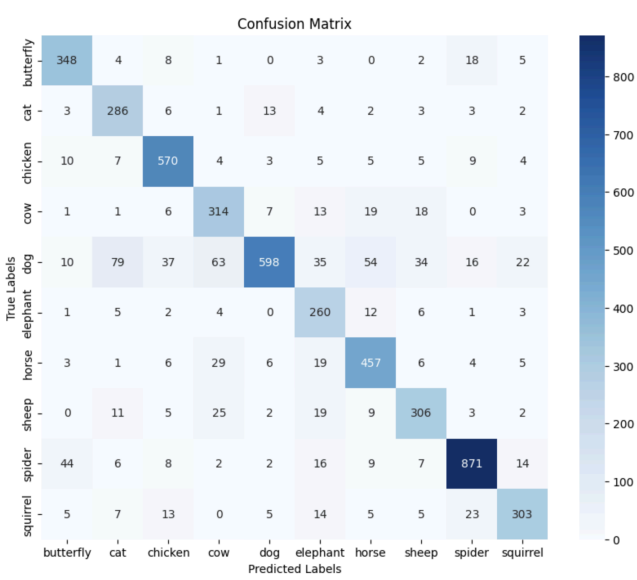
```
    # Block 1 - 32 filters
    layers.Conv2D(32, (3, 3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),

    # Block 2 - 64 filters
    layers.Conv2D(64, (3, 3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2, 2)),

    # Block 3 - 128 filters
    layers.Conv2D(128, (3, 3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),


    # Block 4 - 256 filters
    layers.Conv2D(256, (3, 3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),

    # Block 5 - 256 filters
    layers.Conv2D(256, (3, 3), padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D((2, 2)),

    layers.GlobalAveragePooling2D(),
    layers.Dense(256, activation="relu"),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax")
])
```

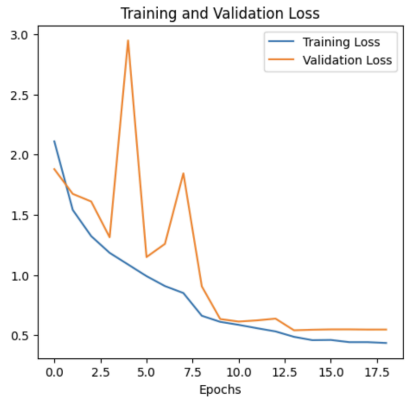
This model gave me a training accuracy of 84.6% and a validation accuracy of 82.4%. I am pretty happy with that.

Model Evaluation

To evaluate my model, I plotted the confusion matrix and looked at the precision, recall, and F1 score.

	precision	recall	f1-score	support
butterfly	0.82	0.89	0.86	389
cat	0.70	0.89	0.78	323
chicken	0.86	0.92	0.89	622
cow	0.71	0.82	0.76	382
dog	0.94	0.63	0.76	948
elephant	0.67	0.88	0.76	294
horse	0.80	0.85	0.82	536
sheep	0.78	0.80	0.79	382
spider	0.92	0.89	0.90	979
squirrel	0.83	0.80	0.82	380
accuracy			0.82	5235
macro avg	0.80	0.84	0.81	5235
weighted avg	0.84	0.82	0.82	5235





I made the following observations:

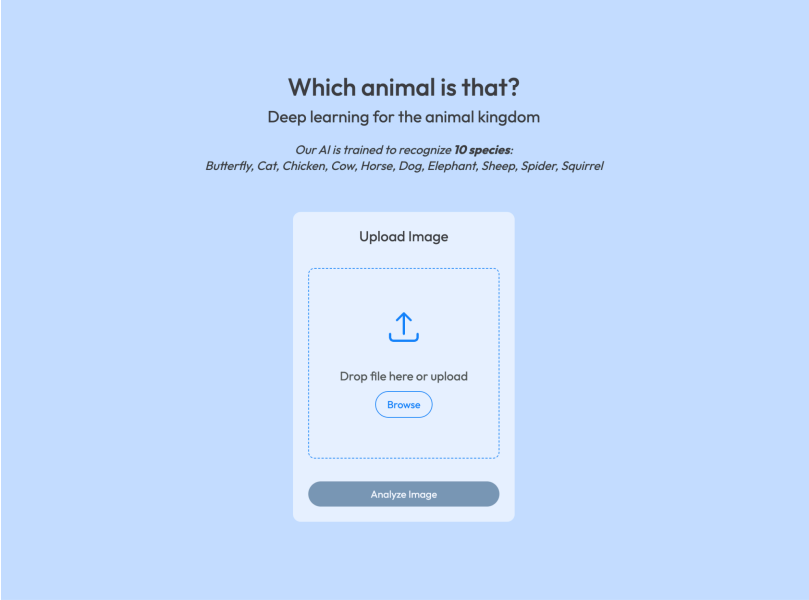
- The precision and recall are better on some classes than others: for example, the model gives best results on the chicken and spider classes, possibly because of their distinct features, such as 8 legs, combs, and beaks
- The macro and weighted averages are very close, which means the model did a good job at handling unbalanced classes
- The precision on cats and recall on dogs are relatively poor, possibly because they are easily confused with other four-legged mammals and also due to the similarity of the backgrounds

Transfer Learning

To improve my accuracy even further, I tested three pre-trained models: MobileNet, VGG16, and Inception.
I achieved significantly higher results with each of them: for each model, my accuracy on the validation set was between 95% and 97%. If I were to use one of these models in my project, I would pick MobileNet, since it turned out to be the most lightweight (13.6 MB).
However, for this project, I decided to keep my custom model.

Application

I built my application using Flask, which allowed me to implement a custom UI.


Which animal is that?
Deep learning for the animal kingdom
Our AI is trained to recognize 10 species:
Butterfly, Cat, Chicken, Cow, Horse, Dog, Elephant, Sheep, Spider, Squirrel
Upload Image
Drop file here or upload
Browse
Analyze Image